Search

Gradle User Manual
Getting Started

Gradle Tutorials
- Beginner Tutorial
- Intermediate Tutorial
- Advanced Tutorial

Gradle Releases
All Releases
Release Notes
Installing Gradle
- Upgrading Gradle
  - Within versions 9.x.y
  - To version 9.0.0
  - Within versions 8.x
  - From version 7.x to 8.0
  - From version 6.x to 7.0
  - From version 5.x to 6.0
  - From version 4.x to 5.0
- Migrating to Gradle
Compatibility Notes
Gradle's Feature Lifecycle

Gradle Fundamentals
- Learning Gradle Basics
- Writing Build Scripts
- Creating Plugins

Gradle Reference
- Runtime and Configuration
- DSLs and APIs
- Gradle Managed Types
- Tasks
- Plugins
- Dependencies
- Platforms
- Best Practices
- Other Topics

Structuring Gradle Builds
Organizing Projects
Multi-Project Builds
Sharing Build Logic
Composite Builds
Configuration on Demand
Isolated Projects

Optimizing Gradle Builds
Improving Performance
- Build Cache
- Configuration Cache

Securing Gradle Builds
Supply Chain Security
Verifying Dependencies

IDE & Tool Integration
Third-party Tools
- APIs

How-To-Guides
- Structuring Builds
- Dependency Management

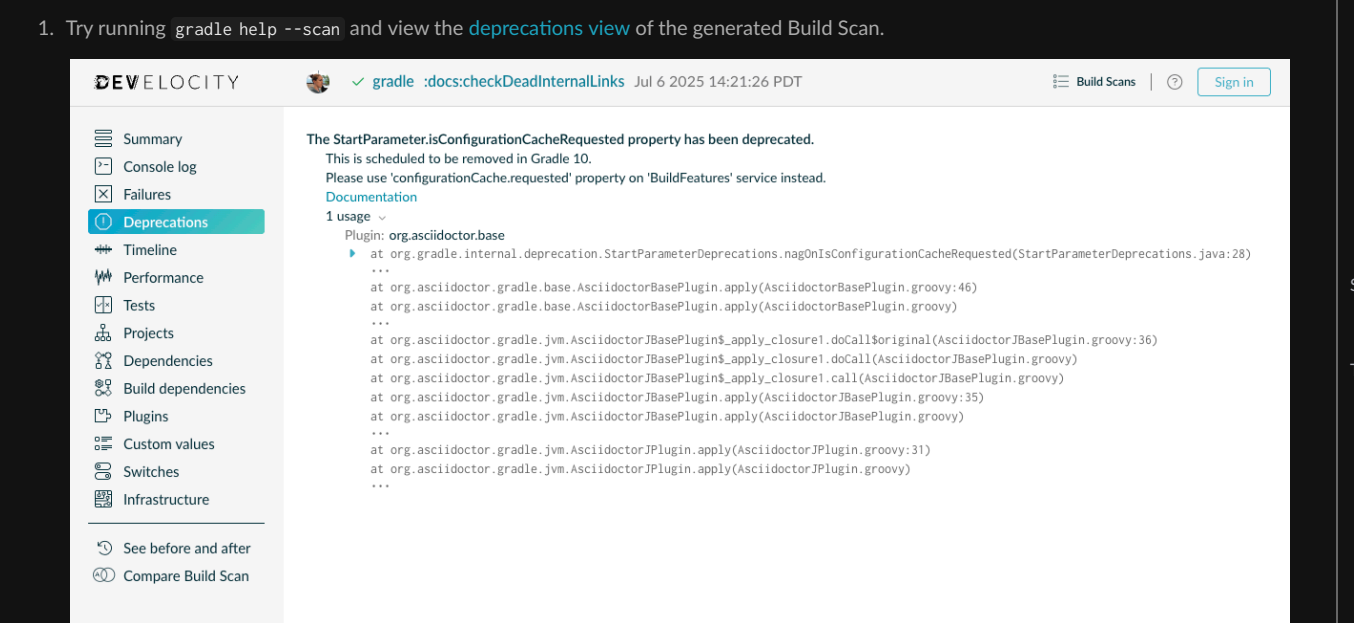
More Resources
Samples
Glossary
Single Page Version

Upgrading to Gradle 9.0.0

This chapter provides the information you need to migrate your Gradle 8.14.4 builds to Gradle 9.0.0. For migrating within Gradle 8.x, see the older migration guide first.

We recommend the following steps for all users:

1. Try running `gradle help --scan` and view the deprecations view of the generated Build Scan.



The StartParameter.isConfigurationCacheRequested property has been deprecated.
This is scheduled to be removed in Gradle 10.
Please use 'configurationCache.requested' property on 'BuildFeatures' service instead.
Documentation
1 usage
Plugin: org.asciidoctor.base

```
at org.gradle.internal.deprecation.StartParameterDeprecations.nagOnIsConfigurationCacheRequested(StartParameterDeprecations.java:28)
...
at org.asciidoctor.gradle.base.AsciidoctorBasePlugin.apply(AsciidoctorBasePlugin.groovy:46)
at org.asciidoctor.gradle.base.AsciidoctorBasePlugin.apply(AsciidoctorBasePlugin.groovy)
...
at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin$_apply_closure1.doCall$original(AsciidoctorJBasePlugin.groovy:36)
at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin$_apply_closure1.doCall(AsciidoctorJBasePlugin.groovy)
at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin$_apply_closure1.call(AsciidoctorJBasePlugin.groovy)
at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin.apply(AsciidoctorJBasePlugin.groovy:35)
at org.asciidoctor.gradle.jvm.AsciidoctorJBasePlugin.apply(AsciidoctorJBasePlugin.groovy)
...
at org.asciidoctor.gradle.jvm.AsciidoctorJPlugin.apply(AsciidoctorJPlugin.groovy:31)
at org.asciidoctor.gradle.jvm.AsciidoctorJPlugin.apply(AsciidoctorJPlugin.groovy)
...
```

   This lets you see any deprecation warnings that apply to your build.

   Alternatively, you can run `gradle help --warning-mode=all` to see the deprecations in the console, though it may not report as much detailed information.

2. Update your plugins.

   Some plugins may break with a new major version of Gradle, as these releases can remove public APIs. Using the latest version of a plugin increases the likelihood that it is already compatible with the new major Gradle version.

3. Run `gradle wrapper --gradle-version 9.0.0` to update the project to 9.0.0.
4. Try to run the project and debug any errors using the Troubleshooting Guide.

Runtime requirements and DSL changes

Java Virtual Machine (JVM) 17 or higher is required

Gradle 9.0.0 requires a Java Virtual Machine (JVM) version 17 or higher to run the Gradle Daemon. This is a breaking change from previous versions, which supported JVM 8 and higher.

Your build can still target lower JVM versions using Toolchains for compilation, testing and other workers (Checkstyle, Javadoc, etc).

The Gradle wrapper and command-line launcher can run with JVM 8, but it still requires a newer JVM to start the build. For more, see the Running Gradle on older JVMs section below.

Gradle Tooling API and TestKit remain compatible with JVM 8 and higher.

Upgrade to Kotlin 2.2.0

Gradle now embeds Kotlin 2.2.0, upgrading from the previously embedded version 2.0.21.

For full details and potential breaking changes, consult the Kotlin release notes.

Kotlin DSL and plugins use the Kotlin language version 2.2

The Kotlin DSL has been upgraded to the latest stable Kotlin 2.2.x runtime and uses Kotlin language version 2.2 across the entire toolchain. This marks a shift from Gradle 8.x, which embedded Kotlin 2.0 starting in 8.11 but continued using Kotlin language version 1.8 for compatibility.

This change impacts not only Kotlin DSL scripts (`.gradle.kts`) but also build logic and plugins written in Kotlin (both classic and convention plugins). Users should review their code for compatibility with Kotlin 2.2, as both Kotlin 2.0 and Kotlin 2.1 introduced several breaking language changes.

Due to changes in how the Kotlin 2 compiler handles script compilation, you can no longer refer to the script instance using labels like `this@Build_gradle`, `this@Settings_gradle`, or `this@Init_gradle`. If they are used to reference the DSL script target object, then use `project`, `settings`, or `gradle` instead. If they are used to reference a top-level symbol that happens to have the same name as a nested symbol, then use a different name, possibly by adding an intermediary variable.

This upgrade also includes an important change for build-logic and plugin authors: support for Kotlin language versions from 1.4 to 1.7 has been removed.

Read the JSpecify section below to learn about potential breaking changes in Kotlin build logic code related to nullability.

Upgrade to Groovy 4.0.27

Groovy has been upgraded from version 3.0.24 to 4.0.27. The update to Groovy 4 comes with many breaking changes, such as the removal of legacy packages, changes in the module structure, a parser rewrite, and bytecode output changes. For a complete overview of Groovy language changes from 3.x to 4.x, see the Groovy 4.0 release notes. These changes may affect those who use Groovy directly or indirectly in Gradle, and in rare cases, users relying on transitive dependencies.

> **TIP**
> Popular Groovy classes that used to be in packages like `groovy.util` are now in different packages to account for the JPMS "split package requirement". See this specific entry.

`is`-prefixed `Boolean` properties no longer recognized by Groovy

Groovy 4 no longer treats getters with an `is` prefix and a `Boolean` return type as properties. Gradle still recognizes these as properties for now, but this behavior will change in Gradle 10 to align with Groovy 4. See the deprecation notice for more details.

`DELEGATE_FIRST` closures may now prefer the delegate in some cases

Groovy 4 has changed the behavior of closures using the `DELEGATE_FIRST` strategy. Dynamic lookups for properties and methods will now prefer the delegate over the owner. This can result in different behavior when using closures in Gradle scripts, such as certain methods not being found or (e.g. with `.with { }`) some dynamic properties taking precedence over outer properties or methods. Generally, this should not affect Gradle scripts, as Gradle does not use `DELEGATE_FIRST` closures with dynamic properties in its API.

Workarounds include using `@CompileStatic` to avoid dynamic lookups, or explicitly qualifying calls with `owner.`, `this.`, or `super.` as needed.

For full clarity, in Groovy 3, the lookup order was:

1. Delegate's `invokeMethod`, which chooses known methods and does no dynamic lookup.
2. Owner's `invokeMethod`, which chooses known methods and does no dynamic lookup.
3. Delegate's `invokeMissingMethod`, which does dynamic lookup including via properties.
4. Owner's `invokeMissingMethod`, which does dynamic lookup including via properties.

In Groovy 4, the lookup order is:

1. Delegate's `invokeMethod`, which chooses known methods and does no dynamic lookup.
2. Delegate's `invokeMissingMethod`, which does dynamic lookup including via properties.
3. Owner's `invokeMethod`, which chooses known methods and does no dynamic lookup.
4. Owner's `invokeMissingMethod`, which does dynamic lookup including via properties.

Private properties and methods may be inaccessible in closures

Closures defined in a parent class that reference its private properties or methods may no longer have access to them in subclasses. This is due to a Groovy bug that will not be resolved until Groovy 5. This applies to both buildscripts and plugins written in Groovy.

As a workaround, apply `@CompileStatic` to the class to remove the dynamic lookup.

This is tracked by https://github.com/gradle/gradle/issues/32476.

Super methods may be inaccessible from Groovy 3.x code

Groovy 4 changed how `super` method calls are resolved at runtime. As a result, code compiled with Groovy 3.x may be unable to access `super` methods, as the code does not contain the appropriate runtime code to locate them. This only applies to plugins written in Groovy 3.x, from Gradle 8.x and earlier.

Plugin changes

Plugins written with the Kotlin DSL require Gradle >= 8.11

When building and publishing plugins using the Kotlin DSL on Gradle 9.x.x, those plugins will only be usable on Gradle 8.11 or newer. This is because Gradle 8.11 is the first release that embeds Kotlin 2.0 or higher, which is required to interpret Kotlin metadata version 2.

If you want your plugin to remain compatible with older Gradle versions, you must explicitly compile it against an earlier Kotlin version (1.x).

For example, to support Gradle 6.8 and newer, configure your plugin to target Kotlin 1.7 like this:

build.gradle.kts

```kotlin
import org.jetbrains.kotlin.gradle.dsl.KotlinVersion
import org.jetbrains.kotlin.gradle.tasks.KotlinCompile

plugins {
    `kotlin-dsl`
}

tasks.withType<KotlinCompile>().configureEach {
    compilerOptions {
        languageVersion = KotlinVersion.KOTLIN_1_7
        apiVersion = KotlinVersion.KOTLIN_1_7
    }
}
```

Refer to Gradle's compatibility matrix for details on which Kotlin version is embedded in each Gradle release.

> **NOTE**
> Plugins written using the Kotlin DSL and published with Gradle 7.x or 8.x remain compatible with Gradle 6.8 and newer.

Plugins written with the Groovy DSL require Gradle >= 7.0

Plugins authored using the Groovy DSL and built with Gradle 9.x.x require Gradle 7.0 or newer to run. This is because Gradle 7.0 introduced Groovy 3.0 support, and Gradle 9 embeds Groovy 4.0.

Since Gradle 9.0.0 uses Groovy 4.0 internally, plugins built with it may not behave as expected when run on older Gradle versions. For best compatibility, such plugins should be used with Gradle 9.0.0 or later.

> **NOTE**
> Plugins written with the Groovy DSL and published using Gradle 7.x or 8.x remain compatible with Gradle 5.0 and above.

Lowest supported Kotlin Gradle Plugin version change

Starting with Gradle 9.0.0, the minimum supported Kotlin Gradle Plugin version is 2.0.0. Earlier versions are no longer supported as they rely on Gradle APIs that have been removed.

For Gradle 8.x, the minimum supported version was 1.6.10.

Lowest supported Android Gradle Plugin version change

Starting with Gradle 9.0.0, the minimum supported Android Gradle Plugin version is 8.4.0. Earlier versions are no longer supported as they rely on Gradle APIs that have been removed.

For Gradle 8.x, the minimum supported version was 7.3.0.

Lowest supported Gradle Enterprise Plugin version change

Starting with Gradle 9.0.0, the minimum supported Gradle Enterprise Plugin version is 3.13.1. Earlier versions are no longer supported as they rely on Gradle APIs that have been removed.

Consider upgrading to the latest version of the Gradle Enterprise Plugin, or better yet, upgrade to the latest version of the Develocity Plugin.

For Gradle 8.x, the minimum supported version was 3.0.

C++ and Swift plugins no longer depend on software model based plugins

C++ Application Plugin, C++ Library Plugin, Swift Application Plugin, and Swift Library Plugin have been updated and no longer rely on the software model plugin infrastructure.

As a result, `toolChains` should be configured directly at the top-level of your build script instead of within a `model { }` block.

For example, instead of:

build.gradle

```groovy
plugins {
    id("cpp-library")
}

model {
    toolChains {
        clang(Clang) {
            eachPlatform {
                cCompiler.executable = "clang"
                cppCompiler.executable = "clang++"
            }
        }
    }
}
```

This should become:

build.gradle

```groovy
plugins {
    id("cpp-library")
}

toolChains {
    clang(Clang) {
        eachPlatform {
            cCompiler.executable = "clang"
            cppCompiler.executable = "clang++"
        }
    }
}
```

This change needs to be made in builds using the new C++ or Swift plugins or the existing software model based plugins.

Scala plugins no longer create unresolvable configurations

Previously, the Scala plugins used configurations named `incrementalScalaAnalysisFor` to resolve incremental analysis information between projects. However, these configurations were unresolvable and could lead to errors in the `dependencies` report.

As of Gradle 9.0.0, these configurations are no longer created or used by the Scala plugins.

Settings file changes

Project directories must exist and be writeable

Gradle will fail if a project is included that does not correspond to an existing directory on the file system, or if the directory exists but is read-only.

For example, you may see an error like:

```
* What went wrong:
Configuring project ':subproject1' without an existing directory is not allowed. The configured projectDirectory '.../s

* Try:
> Make sure the project directory exists and is writable.
```

See upgrading_version_8.html for more details.

Task changes

`ValidatePlugins` task now requires Java Toolchains

In Gradle 9.0.0, using the `ValidatePlugins` task without applying the Java Toolchains plugin will result in an error.

To fix this, explicitly apply the `jvm-toolchains` plugin:

Kotlin Groovy

build.gradle

```groovy
plugins {
    id 'jvm-toolchains'
}
```

> **TIP**
> The `jvm-toolchains` plugin is automatically applied by the Java Library Plugin and other JVM-related plugins. If you are already applying one of those, no further action is needed.

Archive tasks (Jar, Ear, War, Zip, AbstractArchiveTask) produce reproducible archives by default

> **NOTE**
> This change may affect existing builds that relied on the previous behavior of archive tasks, where file order was not deterministic, and file timestamps and permissions were taken from the file system.

In Gradle 9.0.0, the default behavior of archive tasks (such as `Jar`, `Ear`, `War`, `Zip`, and `AbstractArchiveTask`) has changed to produce reproducible archives by default. That means that:

- File order in the archive is now deterministic.
- Files have fixed timestamps (timestamps depends on the archive type).
- All directories have fixed permissions set to `0755`.
- All files have fixed permissions set to `0644`.

This change improves the reproducibility of builds and ensures that the generated archives are consistent across different environments.

You can restore the previous behaviour for one or more properties for your task with the following configuration:

Kotlin Groovy

build.gradle

```groovy
tasks.withType(AbstractArchiveTask).configureEach {
    // Makes file order non deterministic
    reproducibleFileOrder = false
    // Use file timestamps from the file system
    preserveFileTimestamps = true
    // Use permissions from the file system
    useFileSystemPermissions()
}
```

You can also preserve the file system permissions across archives tasks by configuring a property:

gradle.properties

```properties
org.gradle.archives.use-file-system-permissions=true
```

`Test` tasks may no longer execute expected tests

In previous releases, it was possible to create `Test` tasks without any additional configuration. By convention, Gradle used the classpath and test classes from the `test` source set.

Kotlin Groovy

build.gradle

```groovy
plugins {
    id 'java-library'
}

// configure test dependencies
// ...

tasks.register("otherTest", Test)
```

In this example, `otherTest` relied on the deprecated convention. This scenario started to emit a deprecation warning in 8.1. Gradle would execute the same tests with `otherTest` and the built-in `test` task because Gradle used the classpath and test classes from the `test` source set.

The convention has been removed, but builds will not fail if they were relying on this behavior. Builds that emitted the deprecation warning will silently stop executing tests. In the example above, `otherTest` will be skipped and execute no tests because it has no test classes configured.

To return to the previous behavior, you must explicitly configure the `Test` task:

Kotlin Groovy

build.gradle

```groovy
tasks.register("otherTest", Test) {
    testClassesDirs = testing.suites.test.sources.output.classesDirs
    classpath = testing.suites.test.sources.runtimeClasspath
}
```

or add the extra `Test` task through test suites:

Kotlin Groovy

build.gradle

```groovy
testing {
    suites {
        test {
            targets {
                otherTest
            }
        }
    }
}
```

`test` task fails when no tests are discovered

When test sources are present and no filters are applied, the `test` task will now fail with an error if it runs but doesn't discover any tests. This is to help prevent misconfigurations where the tests are written for one test framework but the test task is mistakenly configured to use another test framework. If filters are applied, the outcome depends on the `failOnNoMatchingTests` property.

This behavior can be disabled by setting the `failOnNoDiscoveredTests` property to `false` in the test task configuration:

Kotlin Groovy

build.gradle

```groovy
tasks.withType(AbstractTestTask).configureEach {
    failOnNoDiscoveredTests = false
}
```

Stale outputs outside the build directory are no longer deleted

In previous versions of Gradle, class files located outside the build directory were deleted when considered stale. This was a special case for class files registered as outputs of a source set.

Because this setup is uncommon and forced Gradle to eagerly realize all compile related tasks in every build, the behavior has been removed in Gradle 9.0.0.

Gradle will continue to clean up stale outputs inside the build directory as needed.

`model` and `component` tasks are no longer automatically added

The `model` and `component` tasks report on the structure of legacy software model objects configured for the project. Previously, these tasks were automatically added to the project for every build. These tasks are now only added to a project when a rule-based plugin is applied (such as those provided by Gradle's support for building native software).

API changes

Gradle API now uses JSpecify nullability annotations

Gradle has supported null safety in its public API since Gradle 5.0, allowing early detection of nullability issues when writing Kotlin build scripts or plugin code in Java or Kotlin.

Previously, Gradle used annotations from the now-dormant JSR-305 to indicate nullability. While useful, JSR-305 had limitations and is no longer actively maintained.

Starting with Gradle 9.0.0, the Gradle API now uses JSpecify annotations. JSpecify provides a modern, standardized set of annotations and semantics for nullability in Java APIs, improving support in IDEs and during compilation.

Because JSpecify's semantics differ slightly from JSR-305, you might see new warnings or errors in your Kotlin or Java plugin code. These typically indicate places where you need to clarify or adjust null handling, and modern compilers and IDEs should provide helpful messages to guide you.

Kotlin 2.1, when combined with JSpecify annotations in the Gradle API, introduces stricter nullability handling. Some formerly-valid code may now fail to compile due to more precise type checking.

Common breaking changes:

- Unbounded generics for types that have generic bounds will now fail to compile.

  For example if you have a Kotlin extension function on `Provider<T>` whose signature is `fun <T> Provider<T>.some()` you must qualify `<T>` as `<T : Any>` because `T` isn't nullable on `Provider<T>`.

- The nullability of generic bounds is now handled strictly.

  For example, you can't use `Property<String?>` anymore because the `T` in `Property<T>` is not nullable.

  Another example is using a function from the Gradle API that takes a `Map<String, *>` parameter ; you could pass a map with nullable values before, you can't do that anymore.

> **NOTE**
> Plugins that use `javax.annotation` (JSR-305) annotations will continue to work in Gradle 9.0.0 as they did before.

Methods on public API types made final

The methods `AndSpec.and` and `GenerateBuildDashboard.aggregate` have been declared `final` to support the use of the `@SafeVarargs` annotation.

These types were not intended to be subclassed. However, if your build logic or a plugin attempts to override these methods, it will now result in a runtime failure.

Injection getters are now `abstract`

All Gradle-provided classes that have `@Inject` annotated getters now have those getters declared as `abstract`. This will require all classes that extend Gradle-provided classes to be `abstract`.

`ConfigurationVariant.getDescription` is now a `Property<String>`

This method was added in Gradle 7.5 and was previously a `Optional<String>`. This property was not configurable by public APIs.

By making the description a `Property<String>`, secondary variants have a user configurable description that appears in the `outgoingVariants` report.

New subtypes of `ComponentIdentifier` introduced

Gradle 9.0.0 introduces `RootComponentIdentifier`, a new subtype of `ComponentIdentifier`.

APIs which return instances of `ComponentIdentifier` may now return identifier instances of this new type. For example, the `ComponentResult`, `ResolvedVariantResult`, and `ArtifactView` APIs, among others, are affected.

In future Gradle versions, additional subtypes of `ComponentIdentifier` may be introduced. Build logic should remain resilient to unknown `ComponentIdentifier` subtypes returned by Gradle APIs.

Packaging and artifact behavior changes

Artifact Signing now matches OpenPGP Key Version

Starting with Gradle 9.0.0, the `signing` plugin produces OpenPGP signatures that match the version of the key used. This change ensures compliance with RFC 9580 and introduces support for OpenPGP version 6 keys. Previously, Gradle always generated OpenPGP version 4 signatures, regardless of the key version.

Ear and War plugins build all artifacts with `assemble`

Prior to Gradle 9.0.0, applying multiple packaging plugins (e.g., `ear`, `war`, `java`) to the same project resulted in special behavior where only one artifact type was built during `assemble`. For example:

- Applying the `ear` plugin would skip building `war` and `jar` artifacts.
- Applying the `war` plugin would skip building the `jar`.

This special handling has been removed in Gradle 9.0.0. Now, if multiple packaging plugins are applied, all corresponding artifacts will be built when running the `assemble` task. For example, a project applying the `ear`, `war`, and `java plugins` will now produce `.ear`, `.war`, and `.jar` files during `assemble`.

Ear and War plugins contribute all artifacts to the `archives` configuration

In previous versions of Gradle, applying multiple packaging plugins (`ear`, `war`, `java`) resulted in selective behavior for the `archives` configuration. For example:

- Applying the `ear` plugin excluded `jar` and `war` artifacts from archives.
- Applying the `war` plugin excluded the `jar` artifact from archives.

This behavior has been removed in Gradle 9.0.0. Now, when multiple packaging plugins are applied, all related artifacts—EAR, WAR, and JAR—are included in the `archives` configuration.

Gradle no longer implicitly builds certain artifacts during `assemble`

In previous versions of Gradle, the `assemble` task would implicitly build artifacts from any configuration where the `visible` flag was not set to `false`. This behavior has been removed in Gradle 9.0.0.

If you have a custom configuration and want its artifact to be built as part of `assemble`, you now need to explicitly declare the dependency between the artifact and the `assemble` task:

Kotlin Groovy

build.gradle

```groovy
def specialJar = tasks.register("specialJar". Jar) {
    from("foo")
}

def special = configurations.create("special") {
    // In previous versions, this would have been enough to build the specialJar
    // artifact when running assemble
    outgoing.artifact(specialJar)
}

// In Gradle 9.0.0, you need to add a dependency from the artifact to the assemble task
tasks.named("assemble") {
    dependsOn(special.artifacts)
}
```

Gradle no longer implicitly adds certain artifacts to the `archives` configuration

In previous versions of Gradle, the `archives` configuration would automatically include artifacts from any configuration where the `visible` flag was not set to `false`. This implicit behavior has been removed in Gradle 9.0.0.

To include a custom artifact in the `archives` configuration, you must now add it explicitly:

Kotlin Groovy

build.gradle

```groovy
def specialJar = tasks.register("specialJar". Jar) {
    from("foo")
}

configurations {
    create("special") {
        // In previous versions, this would have been enough to add the specialJar
        // artifact to the archives configuration
        outgoing.artifact(specialJar)
    }
    // In Gradle 9.0.0, you need to explicitly add the artifact to the archives
    // configuration
    named("archives") {
        outgoing.artifact(specialJar)
    }
}
```

Gradle Module Metadata can no longer be modified after an eagerly created publication is created from the same component

This behavior previously caused a warning: `Gradle Module Metadata is modified after an eagerly populated publication.`

It will now fail with an error, suggesting a review of the relevant documentation.

Configuration Cache changes

On this Page

Runtime requirements and DSL changes
  Java Virtual Machine (JVM) 17 or higher is required
  Upgrade to Groovy 4.0.27
Plugin changes
  Plugins written with the Kotlin DSL require Gradle >= 8.11
  Plugins written with the Groovy DSL require Gradle >= 7.0
  Lowest supported Kotlin Gradle Plugin version change
  Lowest supported Android Gradle Plugin version change
  Lowest supported Gradle Enterprise Plugin version change
  C++ and Swift plugins no longer depend on software model based plugins
  Scala plugins no longer create unresolvable configurations
Settings file changes
  Project directories must exist and be writeable
Task changes
  `ValidatePlugins` task now requires Java Toolchains
  `Test` tasks may no longer execute expected tests
  `test` task fails when no tests are discovered
  Stale outputs outside the build directory are no longer deleted
  `model` and `component` tasks are no longer automatically added
API changes
  Gradle API now uses JSpecify nullability annotations
  Methods on public API types made final
  Injection getters are now `abstract`
  `ConfigurationVariant.getDescription` is now a `Property<String>`
  New subtypes of `ComponentIdentifier` introduced
Packaging and artifact behavior changes
  Artifact Signing now matches OpenPGP Key Version
  Ear and War plugins build all artifacts with `assemble`
  Ear and War plugins contribute all artifacts to the `archives` configuration
  Gradle no longer implicitly builds certain artifacts during `assemble`
  Gradle no longer implicitly adds certain artifacts to the `archives` configuration

Configuration Cache changes

Unsupported build event listeners are now configuration cache problems

The build event listener registration method `BuildEventsListenerRegistry.onTaskCompletion` accepts arbitrary providers of any `OperationCompletionListener` implementations. However, only providers returned from `BuildServiceRegistry.registerIfAbsent` or `BuildServiceRegistration.getService` are currently supported when the Configuration Cache is enabled.

Previously, unsupported providers were silently discarded and never received events when the configuration cache was used. Starting with Gradle 9.0.0, registering such providers now causes a configuration cache problem and fails the build.

If your build was previously working with the configuration cache (e.g., the listeners were nonessential), you can temporarily revert to the old behavior by setting:

```properties
org.gradle.configuration-cache.unsafe.ignore.unsupported-build-events-listeners=true
```

This property will be removed in Gradle 10.

Configuration Cache entry is now always discarded for incompatible tasks in warning mode

Configuration Cache enables gradual migration by allowing to explicitly mark tasks as incompatible. When incompatible tasks are scheduled for execution, cache entry is not stored and tasks do not run in parallel to ensure correctness. Configuration Cache can also run in the warning mode by enabling `org.gradle.configuration-cache.problems=warn`, which hides problems and allows storing and loading of the cache entry, running tasks in parallel with potential issues. The warning mode exists as a migration and troubleshooting aid and is not intended as a persistent way of ignoring incompatibilities.

Starting with Gradle 9.0.0, the presence of incompatible tasks in the work graph results in the cache entry being discarded regardless of the warning mode to ensure correctness. If you relied on the warning mode previously, consider marking the offending tasks as incompatible instead. However, at this stage of Configuration Cache maturity and ecosystem adoption, we recommend resolving the incompatibilities instead to ensure performance benefits this feature brings.

Updated versions

Upgraded default versions of code quality tools

The default version of Checkstyle is 10.24.0.

The default version of CodeNarc is 3.6.0.

The default version of Pmd is 7.13.0

Upgraded default versions of testing frameworks

When using test suites, the version of several testing frameworks has changed.

The default version of JUnit Jupiter is 5.12.2.

The default version of TestNG is 7.11.0.

The default version of Spock is 2.3.

Upgraded version of Eclipse JGit

Eclipse JGit has been updated from 5.13.3 to 7.2.1.

This update reworks how Gradle configures JGit for SSH operations and introduces support for using the SSH Agent, leveraging the new capabilities available in JGit's SSH agent integration.

Removed APIs and features

Removal of deprecated `jcenter()`

The `jcenter()` repository API has been removed in Gradle 9.0.0. The API was deprecated in Gradle 7.0.

The JCenter repository was redirected to Maven Central in August 2024.

`RepositoryHandler.mavenCentral()` is the closest direct replacement.

Removal of deprecated `JvmVendorSpec.IBM_SEMERU`

The deprecated `JvmVendorSpec.IBM_SEMERU` constant has been removed. Its usage should be replaced by `JvmVendorSpec.IBM`.

Removal of `GroovySourceSet` and `ScalaSourceSet` interfaces

The following source set interfaces have been removed in Gradle 9.0.0:

- `org.gradle.api.tasks.GroovySourceSet`
- `org.gradle.api.tasks.ScalaSourceSet`

To configure Groovy or Scala sources, use the plugin-specific Source Directory Sets instead:

- `groovy`: `GroovySourceDirectorySet`
- `scala`: `ScalaSourceDirectorySet`

For example, to configure Groovy sources in a plugin:

```java
GroovySourceDirectorySet groovySources = sourceSet.getExtensions().getByType(GroovySourceDirectorySet.class);
groovySources.setSrcDirs(Arrays.asList("sources/groovy"));
```

Removal of custom build layout options

The ability to specify custom locations for key build files from the command line has been removed in Gradle 9.0.0. The following options, deprecated in Gradle 8.x, are no longer supported:

- `-c`, `--settings-file` — Specify a custom location for the settings file
- `-b`, `--build-file` — Specify a custom location for the build file

In addition, the `buildFile` property on the `GradleBuild` task has been removed. This means it is no longer possible to set a custom build file path via the `GradleBuild` task.

Removal of conventions

The *"convention"* concept—represented by the `org.gradle.api.plugins.Convention` type—has been deprecated since Gradle 8.2 and is now fully removed in Gradle 9.0.0.

Core Gradle plugins that previously registered deprecated conventions have been updated accordingly.

This implies removal of the Conventions API. These have been removed:

- `org.gradle.api.Task.getConvention()`
- `org.gradle.api.Project.getConvention()`
- `org.gradle.api.plugins.Convention`
- `org.gradle.api.internal.HasConvention`

Existing plugins that use these APIs will fail with Gradle 9.0.0+ and should be updated to use the Extensions API instead.

The table below shows which conventions have been removed and how to migrate:

Plugin	Access	Type	Solution
`war`	`project.war`	`WarPluginConvention`	Configure the `war` task directly instead.
`base`	`project.distDirName`, `project.libsDirName`, `project.archivesBaseName`	`BasePluginConvention`	Replaced by `project.base` extension of type `BasePluginExtension`.
`project-report`	`project.projectReports`	`ProjectReportPluginConvention`	Configure the report task (`TaskReportTask`, `PropertyReportTask`, `DependencyReportTask`, `HtmlDependencyReportTask`) directly.
`ear`	`project.ear`	`EarPluginConvention`	Configure the `ear` task directly instead.

Removal of `org.gradle.cache.cleanup`

The `org.gradle.cache.cleanup` property, which previously allowed users to disable automatic cache cleanup, has been removed in Gradle 9.0.0.

This property no longer has any effect. To control cache cleanup behavior in Gradle 9.0.0 and later, use an init script instead.

Removal of `buildCache.local.removeUnusedEntriesAfterDays`

In Gradle 9.0.0, the property `buildCache.local.removeUnusedEntriesAfterDays` has been removed.

This property was previously used to configure the retention period for the local build cache.

To configure retention for unused entries in the local build cache, use the Gradle User Home cache cleanup settings instead.

Removal of deprecated `org.gradle.util` members

The following members of the `org.gradle.util` package have been removed:

- `CollectionUtils`
- `ConfigureUtil`, `ClosureBackedAction`

  These classes used to provide utilities related to `groovy.lang.Closure`. Plugins should avoid relying on Groovy specifics, such as `Closure`, in their APIs. Instead, plugins should create methods that use `Action`:

  ```java
  abstract class MyExtension {
      // ...
      public void options(Action<? extends MyOptions>  action) {
          action.execute(options)
      }
  }
  ```

  Gradle automatically generates a `Closure`-taking method at runtime for each method with an `Action` as a single argument as long as the object is created with `ObjectFactory#newInstance`.

  As a last resort, to apply some configuration represented by a Groovy Closure, a plugin can use `Project#configure`.

Removal of deprecated `testSourceDirs` and `testResourceDirs` from `IdeaModule`

The deprecated `testSourceDirs` and `testResourceDirs` properties have been removed from `org.gradle.plugins.ide.idea.model.IdeaModule`. This change does not affect the `org.gradle.tooling.model.idea.IdeaModule` type used in the Tooling API. Use the `testSources` and `testResources` properties instead.

Removal of Unix mode based file permissions

Gradle 9.0.0 removes legacy APIs for specifying file permissions using raw Unix mode integers.

A new and more expressive API for configuring file permissions was introduced in Gradle 8.3 and promoted to stable in Gradle 8.8. See:

- `FilePermissions`.
- `ConfigurableFilePermissions`.

The following older methods, deprecated in Gradle 8.8, have now been removed:

- `org.gradle.api.file.CopyProcessingSpec.getFileMode()`
- `org.gradle.api.file.CopyProcessingSpec.setFileMode(Integer)`
- `org.gradle.api.file.CopyProcessingSpec.getDirMode()`
- `org.gradle.api.file.CopyProcessingSpec.setDirMode(Integer)`
- `org.gradle.api.file.FileTreeElement.getMode()`
- `org.gradle.api.file.FileCopyDetails.setMode(int)`

Removal of select Groovy modules from the Gradle distribution

Gradle 9.0.0 removes certain Groovy modules from its bundled distribution. They will no longer be available on the classpath or be available via `localGroovy`:

- `groovy-test`
- `groovy-console`
- `groovy-sql`

Removal of `kotlinDslPluginOptions.jvmTarget`

In Gradle 9.0.0, the `kotlinDslPluginOptions.jvmTarget` property has been removed.

This property was previously used to configure the JVM target version for code compiled with the `kotlin-dsl` plugin.

To set the target JVM version, you should now configure a Java Toolchain instead.

Removal of the `gradle-enterprise` plugin block extension in Kotlin DSL

In Kotlin DSL based `settings.gradle.kts` files, you could previously use the `gradle-enterprise` plugin block extension to apply the Gradle Enterprise plugin using the same version bundled with `gradle --scan`:

```kotlin
plugins {
    `gradle-enterprise`
}
```

This shorthand had no equivalent in the Groovy DSL (`settings.gradle`) and has now been removed.

Gradle Enterprise has been renamed to Develocity, and the plugin ID has changed from `com.gradle.enterprise` to `com.gradle.develocity`. As a result, you must now apply the plugin explicitly using its full ID and version:

```kotlin
plugins {
    id("com.gradle.develocity") version "4.0.2"
}
```

If you're still using the legacy name, you may apply the deprecated plugin ID to ease the transition:

```kotlin
plugins {
    id("com.gradle.enterprise") version "3.19.2"
}
```

We strongly encourage users to adopt the latest released version of the Develocity plugin, even when using it with older versions of Gradle.

Removal of eager artifact configuration accessors in Kotlin DSL

In Gradle 5.0, the type of configuration accessors changed from `Configuration` to `NamedDomainObjectProvider<Configuration>` to support lazy configuration. To maintain compatibility with plugins compiled against older Gradle versions, the Kotlin DSL provided eager accessor extensions such as:

```kotlin
configurations.compileClasspath.files // equivalent to configurations.compileClasspath.get().files
configurations.compileClasspath.singleFile // equivalent to configurations.compileClasspath.get().singleFile
```

These eager accessors were deprecated and removed from the public API in Gradle 8.0 but remained available for plugins compiled against older Gradle versions.

In Gradle 9.0.0, these legacy methods have now been fully removed.

Removal of `libraries` and `bundles` from version catalogs in the `plugins {}` block in Kotlin DSL

In Gradle 8.1, accessing `libraries` or `bundles` from dependency version catalogs within the `plugins {}` block of a Kotlin DSL script was deprecated.

In Gradle 9.0.0, this support has been fully removed. Attempting to reference `libraries` or `bundles` in the `plugins {}` block will now result in a build failure.

Removal of `"name"()` task reference syntax in Kotlin DSL

In Gradle 9.0.0, referencing tasks or other domain objects using the `"name"()` syntax in Kotlin DSL has been removed.

Instead of using `"name"()` to reference a task or domain object, use `named("name")` or one of the other supported notations.

Removal of `outputFile` in `WriteProperties` task

The `outputFile` property in the `WriteProperties` task has been removed in Gradle 9.0.0.

This property was deprecated in Gradle 8.0 and was replaced with the `destinationFile` property.

Removal of `Project#exec`, `Project#javaexec`, and script-level counterparts

The following helper methods for launching external processes were deprecated in Gradle 8.11 and have now been removed in Gradle 9.0.0:

- `org.gradle.api.Project#exec(Closure)`
- `org.gradle.api.Project#exec(Action)`
- `org.gradle.api.Project#javaexec(Closure)`
- `org.gradle.api.Project#javaexec(Action)`
- `org.gradle.api.Script#exec(Closure)`
- `org.gradle.api.Script#exec(Action)`
- `org.gradle.api.Script#javaexec(Closure)`
- `org.gradle.api.Script#javaexec(Action)`
- `org.gradle.kotlin.dsl.InitScriptApi#exec(Action)`
- `org.gradle.kotlin.dsl.InitScriptApi#javaexec(Action)`
- `org.gradle.kotlin.dsl.KotlinScript#exec(Action)`
- `org.gradle.kotlin.dsl.KotlinScript#javaexec(Action)`
- `org.gradle.kotlin.dsl.SettingsScriptApi#exec(Action)`
- `org.gradle.kotlin.dsl.SettingsScriptApi#javaexec(Action)`

Removal of unused public APIs

The following types have been removed in Gradle 9.0.0. These types are not used in Gradle's public API and as such are not useful.

- `org.gradle.api.artifacts.ArtifactIdentifier`
- `org.gradle.api.publish.ivy.IvyDependency`
- `org.gradle.api.publish.maven.MavenDependency`

Stay `UP-TO-DATE` on new features and news:

By entering your email, you agree to our Terms and Privacy Policy.

Email address Subscribe

© 2025 Gradle, Inc. Gradle®, Develocity®, Build Scan®, and the Gradlephant logo are registered trademarks of Gradle, Inc.